

Advanced Java - Master Notes

A Complete Reference for Interview Preparation, Production Work, Real-World Projects, and System Understanding.

Java Full Stack Master Course

Table of Contents

ADVANCED JAVA — MASTER NOTES	6
A Complete Reference Book for Interviews, Production Work & System Understanding	6
TOPIC 1: ENTERPRISE JAVA OVERVIEW	6
1. Introduction	6
1.1 What is it?	6
1.2 Why was it introduced	6
1.3 Historical Background	7
1.4 Interview Definition	7
1.5 Simple Definition	7
2. Real Life Analogy	7
3. The Multi-Tier Architecture Model	8
4. Key Enterprise Java Specifications (Historical Map)	8
5. Application Servers vs Embedded Servers — A Critical Modern Shift	9
6. Code Example — The Simplest Possible Servlet (A Taste of What's Coming)	9
7. Edge Cases / Common Points of Confusion	10
8. Common Mistakes	10
9. Best Practices (For This Course's Trajectory)	10
10. Production Usage	10
11. Frequently Asked Interview Questions	11
12. Revision Notes	11
13. Homework	11
14. Final Summary	12
TOPIC 2: MVC ARCHITECTURE	12
1. Introduction	12
2. The Three Components	13
3. Real Life Analogy	13
4. Request Flow Diagram	13
5. Code Example — A Minimal MVC Flow (Classic Servlet Style)	14
6. MVC in Modern Spring Boot (Preview)	14

7. Edge Cases / Common Points of Confusion	15
8. Common Mistakes	15
9. Best Practices	15
10. Production Usage	15
11. Frequently Asked Interview Questions	15
12. Revision Notes	16
13. Homework	16

TOPIC 3: WEB APPLICATION BASICS 16

1. Introduction	16
2. The HTTP Request-Response Cycle (Foundational Recap)	17
3. Anatomy of a Java Web Application (WAR structure)	17
4. web.xml — The Classic Deployment Descriptor	17
5. Real Life Analogy	17
6. Edge Cases	18
7. Common Mistakes	18
8. Best Practices	18
9. Frequently Asked Interview Questions	18

TOPIC 4: SESSION MANAGEMENT 18

1. Introduction	19
2. How HttpSession Works Internally	19
3. Code Example	19
4. Key HttpSession Methods	20
5. Real Life Analogy	20
6. Edge Cases	20
7. Common Mistakes	20
8. Best Practices	20
9. Production Usage	21
10. Frequently Asked Interview Questions	21

TOPIC 5: COOKIES 21

1. Introduction	21
---------------------------	----

2. How Cookies Work	22
3. Code Example — Creating and Reading Cookies	22
4. Key Cookie Attributes	22
5. HttpSession vs Cookies — How They Relate	22
6. Real Life Analogy	23
7. Edge Cases	23
8. Common Mistakes	23
9. Best Practices	23
10. Frequently Asked Interview Questions	23

TOPIC 6: FILTERS **24**

1. Introduction	24
2. The Filter Chain	24
3. Code Example — A Logging Filter	24
4. Real Life Analogy	24
5. Common Uses of Filters	25
6. Edge Cases	25
7. Common Mistakes	25
8. Best Practices	25
9. Production Usage	26
10. Frequently Asked Interview Questions	26

TOPIC 7: LISTENERS **26**

1. Introduction	26
2. Key Listener Interfaces	26
3. Code Example — Application Startup/Shutdown Listener	27
4. Code Example — Tracking Active Sessions (Common Real Use Case)	27
5. Real Life Analogy	27
6. Filters vs Listeners — Comparison Table	28
7. Edge Cases	28
8. Common Mistakes	28
9. Best Practices	28
10. Production Usage	28

11. Frequently Asked Interview Questions	28
12. Revision Notes (Topics 3-7 Consolidated)	29
13. Homework	29
FINAL SUMMARY — ADVANCED JAVA COMPLETE	29

ADVANCED JAVA — MASTER NOTES

A Complete Reference Book for Interviews, Production Work & System Understanding

TOPIC 1: ENTERPRISE JAVA OVERVIEW

1. Introduction

1.1 What is it?

"Enterprise Java" refers to the ecosystem of specifications, APIs, and frameworks built ON TOP of Core Java (Java SE) specifically to build large-scale, multi-tier, business-critical applications — web applications, distributed systems, transactional business logic — as opposed to standalone desktop/CLI programs. Historically standardized as **J2EE**, then **Java EE**, and now **Jakarta EE** (since Oracle transferred governance to the Eclipse Foundation in 2017).

1.2 Why was it introduced

Core Java (`java.lang`, `java.util`, `java.io`) gives you the LANGUAGE and basic building blocks, but says nothing about how to:

- Serve HTTP requests from thousands of concurrent users.
- Manage database transactions safely across multiple operations.
- Handle user sessions across stateless HTTP requests.
- Structure large codebases so business logic, data access, and presentation don't become tangled.

Enterprise Java standardizes SOLUTIONS to these recurring, universal problems, so every company doesn't reinvent (and re-debug) the same web-server/transaction/session infrastructure from scratch.

1.3 Historical Background

Year	Milestone
1999	J2EE 1.2 released — first standardized enterprise Java platform (Servlets, JSP, EJB).
2003	J2EE 1.4 — Web Services support added.
2006	Renamed Java EE 5 — introduced annotations, simplified EJB 3.0, JPA (Java Persistence API) born.
2009	Java EE 6 — Introduced CDI (Contexts and Dependency Injection), Servlet 3.0.
2017	Oracle transfers Java EE governance to the Eclipse Foundation ; renamed Jakarta EE (due to Oracle retaining trademark rights over the "Java EE" name).
2019+	Jakarta EE 8, 9, 10 — Jakarta EE 9 changed ALL package namespaces from <code>javax.*</code> to <code>jakarta.*</code> — a major, disruptive migration for the entire ecosystem.
Present	Most NEW enterprise Java development happens via Spring Framework/Spring Boot (a non-standard but dominant, more lightweight alternative) rather than full Jakarta EE application servers — though Jakarta EE specifications (Servlets, JPA, CDI) still underpin much of what Spring builds upon.

1.4 Interview Definition

"Enterprise Java refers to the set of specifications and APIs — historically J2EE/Java EE, now Jakarta EE — that extend Java SE with standardized solutions for multi-tier, transactional, web-facing business applications, including Servlets, JSP, EJB, JPA, and CDI, traditionally deployed to a compliant Application Server (WebLogic, WebSphere, JBoss/WildFly)."

1.5 Simple Definition

Enterprise Java is the extra toolkit on top of regular Java specifically for building big business web applications that many people use at once, safely and reliably.

2. Real Life Analogy

- **Bank:** Core Java is like knowing how to do arithmetic and follow logical steps (the raw skill). Enterprise Java is like having an ENTIRE standardized banking-operations rulebook (how to process a thousand simultaneous transactions safely, how to remember which customer is "logged in," how to roll back a failed transfer) — a whole additional layer of infrastructure a single bank teller's arithmetic skill alone doesn't provide.
 - **Restaurant:** Core Java is knowing how to cook a dish. Enterprise Java is the entire restaurant CHAIN's standardized operating system — how orders flow from front-of-house to kitchen to billing, how simultaneous orders from 200 tables don't get mixed up, how a customer's table session persists as they order multiple courses.
-

3. The Multi-Tier Architecture Model

```
+-----+ +-----+ +-----+
| PRESENTATION TIER | | BUSINESS TIER | | DATA TIER |
| (Web browser, | <-> | (Servlets, EJB, | <-> | (Database, via |
| JSP/HTML/REST | | Business Logic) | | JPA/JDBC) |
| responses) | | | |
+-----+ +-----+ +-----+
```

- **Presentation tier:** Handles user interaction — HTML rendering (JSP, or modern REST + a separate frontend like React).
- **Business tier:** Contains the actual application/business logic (validation, calculations, orchestration) — traditionally EJBs, now typically plain Spring `@Service` classes.
- **Data tier:** Manages persistence — traditionally raw JDBC, now typically JPA/Hibernate (see the dedicated Hibernate topic later in this course).

4. Key Enterprise Java Specifications (Historical Map)

Specification	Purpose	Modern Status
Servlets	Handle raw HTTP requests/responses	Still foundational — Spring MVC builds on top of the Servlet API
JSP (JavaServer Pages)	Server-side HTML templating	Largely superseded by REST APIs + modern frontend frameworks (React, Angular)
EJB (Enterprise JavaBeans)	Business logic components with built-in transaction/security management	Largely superseded by lighter-weight Spring <code>@Service</code> beans for new projects
JPA (Java Persistence API)	Object-relational mapping standard	Still very much alive and dominant — implemented by Hibernate, EclipseLink
CDI (Contexts and Dependency Injection)	Standardized dependency injection	Conceptually similar to (and partly inspired) Spring's DI, though Spring uses its own separate DI container
JMS (Java Message Service)	Standardized messaging API	Still used, though Kafka (non-JMS) is now extremely common for high-throughput messaging

5. Application Servers vs Embedded Servers — A Critical Modern Shift

Aspect	Traditional Java EE Application Server	Modern Spring Boot Approach
Deployment model	Build a <code>.war</code> file, DEPLOY it into a pre-installed, separately-managed server (WebLogic, WebSphere, JBoss/WildFly)	Build a self-contained executable <code>.jar</code> with an EMBEDDED server (Tomcat/Jetty/Undertow) bundled INSIDE it
Server management	Separate team/process manages the application server infrastructure	The application manages its own embedded server — "just run the jar"
Startup	Slower, heavier	Fast, lightweight
Modern relevance	Still used in some large, legacy enterprise/regulated environments	The dominant approach for new microservices/cloud-native development

This shift is one of the most important pieces of context for understanding "why does modern Java web development look so different from what a textbook 'Java EE' course might describe" — most new development (and this course's later Spring Boot sections) uses the embedded-server model, NOT traditional application server deployment.

6. Code Example — The Simplest Possible Servlet (A Taste of What's Coming)

```
import jakarta.servlet.http.*;
import jakarta.servlet.annotation.WebServlet;
import java.io.IOException;

@WebServlet("/hello") // maps this servlet to the URL /hello
public class HelloServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response) throws IOException {
        response.setContentType("text/plain");
        response.getWriter().println("Hello from an Enterprise Java Servlet!");
    }
}
```

This single class, deployed to any Servlet-compliant server, handles every incoming HTTP GET request to `/hello` — the foundational building block underneath EVERYTHING covered in the upcoming Servlets, JSP, and even Spring MVC topics.

7. Edge Cases / Common Points of Confusion

Point of Confusion	Clarification
"Is Java EE the same as Jakarta EE?"	Jakarta EE is the CONTINUATION of Java EE under new (Eclipse Foundation) governance — functionally similar, but with a disruptive <code>javax.*</code> → <code>jakarta.*</code> package rename starting Jakarta EE 9
"Does Spring Boot replace Java EE entirely?"	Not exactly — Spring Boot is a separate, independent framework, but it BUILDS ON TOP OF several Jakarta EE specifications (Servlets, JPA) rather than replacing them outright; it replaces the NEED for a traditional application server and heavyweight EJB usage
"Do I need to learn old-style JSP/EJB for modern Java jobs?"	Rarely for GREENFIELD projects (Spring Boot + REST + React dominates), but understanding Servlets/JSP fundamentals remains valuable for legacy system maintenance and for genuinely understanding what Spring MVC does underneath

8. Common Mistakes

- Assuming "Enterprise Java" and "Spring" are the same thing — Spring is a POPULAR, independent framework that happens to build on some Jakarta EE specifications; it is not itself part of the official Jakarta EE standard.
- Confusing Java EE's heavyweight, traditional EJB-based business components with modern, lightweight Spring `@Service` beans — conceptually similar goals, very different implementation weight/complexity.
- Not recognizing that most new job postings expecting "Java EE experience" or "Spring Boot experience" are, practically, asking about overlapping but distinct skill sets.

9. Best Practices (For This Course's Trajectory)

- Understand the FOUNDATIONAL Servlet/JSP concepts covered in the next few topics — they explain WHY modern frameworks (Spring MVC) are structured the way they are, even if you'll rarely write raw Servlets in a real job today.
- Recognize that MVC Architecture (next topic), Session Management, Filters, and Listeners are concepts that persist essentially UNCHANGED in spirit from classic Java EE all the way through to modern Spring Boot — this is genuinely transferable, durable knowledge, not obsolete trivia.

10. Production Usage

- Large, established enterprises (banks, insurance companies, government systems) still run substantial legacy Java EE/Jakarta EE application-server-based systems (WebLogic, WebSphere) that require ongoing maintenance.

- New development at most companies (startups AND large tech companies alike) predominantly uses Spring Boot with embedded servers, deployed as containerized microservices — the direction this entire course builds toward.

11. Frequently Asked Interview Questions

- 1 **What is Enterprise Java / Java EE / Jakarta EE?** A set of specifications extending Core Java with standardized solutions (Servlets, JPA, CDI, EJB) for building multi-tier, transactional business applications.
- 2 **What's the difference between Java EE and Jakarta EE?** Jakarta EE is the continuation of Java EE under the Eclipse Foundation's governance (since 2017), with a major `javax.*` → `jakarta.*` package namespace change starting Jakarta EE 9.
- 3 **Is Spring Boot part of the official Jakarta EE standard?** No — it's an independent, separate framework, though it builds upon several Jakarta EE specifications (like the Servlet API and JPA).
- 4 **What's the key deployment difference between a traditional Java EE application and a modern Spring Boot application?** Traditional Java EE apps deploy a `.war` file to a separately-managed application server; Spring Boot apps bundle an EMBEDDED server inside a self-contained, directly-runnable `.jar`.
- 5 **What are the three tiers in the classic multi-tier enterprise architecture?** Presentation tier, Business tier, Data tier.
- 6 **Why has EJB usage declined in favor of Spring?** Spring's dependency injection and `@Service` beans provide similar capabilities (transaction management, business logic organization) with significantly less configuration overhead and complexity than traditional EJBs required.

12. Revision Notes

ENTERPRISE JAVA – CHEAT SHEET

=====

J2EE (1999) -> Java EE (2006) -> Jakarta EE (2017+, Eclipse Foundation)
Jakarta EE 9+ : `javax.*` -> `jakarta.*` package rename (major migration)

MULTI-TIER: Presentation -> Business -> Data

KEY SPECS: Servlets (HTTP handling), JSP (server-side templating, legacy),
EJB (business components, largely superseded by Spring), JPA (ORM,
still dominant), CDI (dependency injection)

MODERN SHIFT: traditional App Server (`.war`, WebLogic/WebSphere) ->
Spring Boot embedded server (`.jar`, Tomcat/Jetty bundled inside)

Spring Boot is INDEPENDENT of Jakarta EE, but BUILDS ON some of its specs
(Servlet API, JPA)

13. Homework

- 1 Research and write 3-4 sentences on why a large bank might still run legacy Java EE application servers today, despite Spring Boot's popularity for new projects.
 - 2 Look up one real, currently-maintained Jakarta EE application server (e.g., WildFly, Payara, Open Liberty) and note its current version and what it's used for.
-

14. Final Summary

- Enterprise Java (J2EE → Java EE → Jakarta EE) is the standardized specification ecosystem for building multi-tier business applications on top of Core Java.
 - The multi-tier model (Presentation → Business → Data) remains the conceptual backbone of virtually all web application architecture, including modern Spring Boot systems.
 - Modern development has largely shifted from heavyweight EJB/application-server deployment toward lightweight Spring Boot with embedded servers — but the underlying concepts (Servlets, sessions, filters) this course covers next remain foundational and directly transferable.
 - Spring Boot is NOT officially part of Jakarta EE, but heavily builds upon several of its specifications.
-

(End of Topic 1: Enterprise Java Overview.)

TOPIC 2: MVC ARCHITECTURE

1. Introduction

What is it? MVC (Model-View-Controller) is an architectural pattern that separates an application into three interconnected components: **Model** (data and business logic), **View** (presentation/UI), and **Controller** (handles input, coordinates Model and View) — preventing the tangled "spaghetti code" that results from mixing data logic, presentation, and request-handling in one place.

Why introduced: Early web applications (raw Servlets or JSP doing everything — database queries, business rules, AND HTML generation all in one file) became unmaintainable as they grew. MVC forces a clean separation of concerns, so a change to the DATABASE schema doesn't require touching HTML, and a change to the PAGE LAYOUT doesn't require touching business logic.

2. The Three Components

Component	Responsibility	Typical Java Representation
Model	Data + business logic + state	POJOs, JPA entities, service classes
View	Presentation/rendering to the user	JSP, Thymeleaf templates, or JSON (for REST APIs consumed by a separate frontend)
Controller	Receives input (HTTP requests), invokes Model logic, selects the View to render	Servlets (classic), or <code>@Controller/@RestController</code> classes (Spring MVC)

3. Real Life Analogy

- **Restaurant:** The **Model** is the kitchen's actual food/recipes/inventory (the real data and logic). The **View** is the menu/plated presentation shown to the customer. The **Controller** is the waiter — takes the customer's order (request), tells the kitchen (Model) what's needed, and brings back the right presentation (View) to the table.
- **Bank:** The **Model** is the actual account data and interest-calculation logic. The **View** is the printed/displayed statement. The **Controller** is the bank teller/ATM software — receives your request ("withdraw \$100"), asks the Model to process it, then shows you the resulting View (updated balance receipt).

4. Request Flow Diagram

```
User's Browser
| (1) HTTP Request: GET /account/123
v
+-----+
| CONTROLLER | (2) parses request, calls Model
+-----+
|
v
+-----+
| MODEL | (3) fetches/processes data (e.g., from database via JPA)
+-----+
|
v
+-----+
| CONTROLLER | (4) selects a VIEW, passes Model data to it
+-----+
|
v
+-----+
| VIEW | (5) renders the final HTML/JSON output
+-----+
|
v
User's Browser <--- (6) HTTP Response
```

5. Code Example — A Minimal MVC Flow (Classic Servlet Style)

```
// MODEL - just data + logic, knows NOTHING about HTTP or HTML
class Account {
    String owner;
    double balance;
    Account(String owner, double balance) { this.owner = owner; this.balance = balance; }
}

// CONTROLLER - handles the request, talks to the Model, forwards to a View
@WebServlet("/account")
public class AccountController extends HttpServlet {
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        Account account = new Account("Alice", 1500.0); // (in reality: fetched from a database)
        request.setAttribute("account", account); // pass Model data to the View
        request.getRequestDispatcher("/accountView.jsp").forward(request, response); // select the VIEW
    }
}

<!-- VIEW (accountView.jsp) - ONLY presentation, reads Model data passed by the Controller -->
<html>
<body>
<h1>Account Owner: ${account.owner}</h1>
<p>Balance: ${account.balance}</p>
</body>
</html>
```

Notice the clean separation: `Account` (Model) has zero knowledge of HTTP/HTML. `AccountController` (Controller) has zero HTML-rendering logic. `accountView.jsp` (View) has zero business logic — it just displays whatever data it's handed.

6. MVC in Modern Spring Boot (Preview)

```
@Controller
public class AccountController {

    @GetMapping("/account/{id}")
    public String getAccount(@PathVariable int id, Model model) {
        Account account = accountService.findById(id); // MODEL logic (delegated to a service)
        model.addAttribute("account", account); // pass data to the VIEW
        return "accountView"; // VIEW name (e.g., a Thymeleaf template)
    }
}
```

The exact SAME conceptual pattern — this is why understanding classic MVC deeply makes Spring MVC (covered later in this course) feel immediately familiar rather than like an entirely new concept.

7. Edge Cases / Common Points of Confusion

Point of Confusion	Clarification
"Isn't a REST API just Controller + Model, no View?"	The "View" in a REST API context is the JSON/XML representation itself — Spring's <code>@RestController</code> essentially merges the View-selection step into directly serializing the Model data, skipping a separate template-rendering step
Putting database queries directly inside a Controller	A common anti-pattern/violation of MVC — the Controller should DELEGATE to Model/Service logic, not contain business/data-access logic itself
Putting business logic inside a JSP View	Also a violation — Views should be as "dumb"/presentation-only as possible

8. Common Mistakes

- Writing database queries or complex business calculations directly inside a Controller class — violates separation of concerns, makes testing and reuse harder.
- Embedding significant Java logic (loops, conditionals beyond simple display logic) directly inside JSP files ("scriptlets") — a well-known JSP anti-pattern (covered further in the JSP topic).
- Confusing MVC with the specific Java EE Servlet/JSP technology — MVC is a GENERAL architectural pattern, applicable (and applied) across many languages/frameworks (Rails, Django, ASP.NET, React+Redux-style patterns), not something unique to Java.

9. Best Practices

- Keep Controllers "thin" — they should orchestrate (parse input, call services, select a view), not contain business logic themselves.
- Keep Models rich with genuine business logic/validation, not just plain data holders (an "anemic domain model" is a well-known anti-pattern).
- Keep Views focused purely on presentation — no business logic, minimal conditional complexity.

10. Production Usage

- Virtually EVERY modern web framework (Spring MVC, Django, Ruby on Rails, ASP.NET MVC, Laravel) is built around this exact MVC pattern (or a close variant like MVVM for rich client-side frameworks).
- Understanding MVC deeply is foundational not just for classic JSP/Servlet work, but for the ENTIRE REST APIs and Spring Boot sections later in this course.

11. Frequently Asked Interview Questions

- 1 **What does MVC stand for, and what does each component do?** Model (data/business logic), View (presentation), Controller (handles input, coordinates Model and View).

- 2 **Why is MVC important for maintainability?** It separates concerns so changes to data logic, presentation, and request handling don't ripple across unrelated code, making the codebase easier to test, modify, and reason about.
- 3 **In a REST API, what serves as the "View"?** The JSON/XML representation of the data itself — Spring's `@RestController` effectively serializes the Model directly instead of rendering a separate HTML template.
- 4 **What's a common anti-pattern that violates MVC?** Placing database queries or complex business logic directly inside a Controller (or JSP View) instead of delegating to proper Model/Service classes.
- 5 **Is MVC specific to Java?** No — it's a general architectural pattern used across virtually all major web frameworks in every popular language.

12. Revision Notes

```
MVC ARCHITECTURE – CHEAT SHEET
=====
MODEL -> data + business logic (knows NOTHING about HTTP/HTML)
VIEW -> presentation/rendering (knows NOTHING about business logic)
CONTROLLER -> receives request, calls Model, selects/populates the View

FLOW: Request -> Controller -> Model -> Controller -> View -> Response

REST API: "View" = the JSON/XML serialization itself (no separate template)

ANTI-PATTERNS: DB queries in Controller, business logic in View/JSP,
"anemic" Models with no real logic (just getters/setters)

SAME PATTERN used by: Spring MVC, Django, Rails, ASP.NET MVC, Laravel
```

13. Homework

- 1 Refactor a hypothetical Servlet that currently does database querying, business calculation, AND HTML generation all in one method, splitting it into proper Model/View/Controller pieces.
- 2 Write 2-3 sentences explaining, in your own words, what "View" means in the context of a REST API that returns only JSON.

(End of Topic 2: MVC Architecture.)

TOPIC 3: WEB APPLICATION BASICS

1. Introduction

What is it? This topic covers the foundational concepts every Java web application relies on: the HTTP request-response cycle, the structure of a deployable web application (WAR), and the `web.xml`/annotation-based deployment descriptor that ties everything together.

2. The HTTP Request-Response Cycle (Foundational Recap)

```
CLIENT (Browser) SERVER (Servlet Container, e.g., Tomcat)
| |
|---- HTTP Request (GET /app/hello) ----->|
| | 1. Container finds the matching Servlet
| | 2. Servlet's doGet()/doPost() executes
| | 3. Response is built
|<--- HTTP Response (status, headers, body) --|
| |
```

(Full HTTP method/status code detail is covered in the dedicated REST APIs topic later in this course — this topic focuses on the WEB APPLICATION structure itself.)

3. Anatomy of a Java Web Application (WAR structure)

```
my-webapp.war
■■■■ WEB-INF/
■ ■■■ web.xml <- deployment descriptor (classic XML configuration)
■ ■■■ classes/ <- compiled .class files
■ ■ ■■■ com/company/app/...
■ ■■■ lib/ <- third-party JAR dependencies
■■■■ index.jsp <- publicly accessible JSP/HTML files
■■■■ css/, js/, images/ <- static resources
```

- Everything inside `WEB-INF/` is NOT directly accessible via a browser URL (protected by the container) — only reachable through Servlet/JSP request handling, a deliberate security boundary.
- Static resources OUTSIDE `WEB-INF/` (like `index.jsp`, `css/`) are directly browser-accessible.

4. `web.xml` — The Classic Deployment Descriptor

```
<web-app>
<servlet>
<servlet-name>HelloServlet</servlet-name>
<servlet-class>com.company.app.HelloServlet</servlet-class>
</servlet>
<servlet-mapping>
<servlet-name>HelloServlet</servlet-name>
<url-pattern>/hello</url-pattern>
</servlet-mapping>
</web-app>
```

Since Servlet 3.0 (Java EE 6, 2009), **annotations** (`@WebServlet("/hello")`) can replace most `web.xml` configuration entirely — modern applications rarely hand-write `web.xml` anymore, though understanding it remains valuable for legacy system maintenance and deep understanding of what annotations are doing under the hood.

5. Real Life Analogy

- **College:** A WAR file is like a sealed course-registration package delivered to a specific university (the Servlet Container) — `WEB-INF/` is like the confidential internal administrative office (not walk-in accessible), while the public course catalog (static resources) is freely browsable by anyone.

6. Edge Cases

Case	Result
Trying to directly access <code>yourSite.com/WEB-INF/web.xml</code> from a browser	Blocked by the container — <code>WEB-INF/</code> contents are never directly served, by design
Both <code>web.xml</code> AND an <code>@WebServlet</code> annotation configure the SAME servlet mapping differently	Behavior depends on the specific container's precedence rules — best practice is to use ONE consistent configuration approach, not mix both

7. Common Mistakes

- Placing sensitive configuration/class files OUTSIDE `WEB-INF/`, accidentally making them directly browser-accessible.
- Mixing `web.xml`-based and annotation-based servlet configuration inconsistently within the same application, causing confusing, hard-to-debug mapping conflicts.

8. Best Practices

- Prefer annotation-based configuration (`@WebServlet`, `@WebFilter`, `@WebListener`) for new code — less verbose, co-located with the actual class.
- Understand `web.xml` regardless, since many legacy/enterprise systems still rely on it, and some configuration (like session timeout defaults) is sometimes still easier to express there.

9. Frequently Asked Interview Questions

- 1 **What is a WAR file?** A packaged, deployable Java web application archive, containing compiled classes, dependencies, and web resources in a standardized directory structure.
- 2 **What's special about the `WEB-INF/` directory?** Its contents (classes, `web.xml`, internal libraries) are NEVER directly accessible via a browser URL — only reachable through the application's own request handling.
- 3 **What replaced most `web.xml` configuration in modern Java web apps?** Annotations (`@WebServlet`, `@WebFilter`, `@WebListener`), introduced in Servlet 3.0 (2009).

TOPIC 4: SESSION MANAGEMENT

1. Introduction

What is it? HTTP is inherently **stateless** — every request is independent, with no built-in memory of previous requests from the same user. Session management is the set of techniques (primarily via `HttpSession`) that let a server remember a specific user's state (login status, shopping cart contents) ACROSS multiple separate HTTP requests.

Why introduced: Without session management, every single page load would require the user to re-authenticate and re-establish all context from scratch — a shopping cart couldn't persist between "add item" and "checkout" requests, since the server would have no memory connecting those two separate, stateless requests as belonging to the SAME user.

2. How `HttpSession` Works Internally

1. User's FIRST request arrives -> server creates a new `HttpSession` object, assigns it a unique SESSION ID.
2. Server sends that Session ID back to the browser, typically via a COOKIE named "JSESSIONID".
3. On EVERY SUBSEQUENT request, the browser automatically sends that same cookie back.
4. The server uses the Session ID to look up the SAME `HttpSession` object (stored server-side, in memory or a distributed cache), retrieving whatever data was previously stored in it.

BROWSER SERVER

```
| |
|--- Request 1 (login) ----->| creates HttpSession (id=ABC123)
|<-- Response + Set-Cookie: JSESSIONID=ABC123 --|
| |
|--- Request 2 (add to cart) |
| Cookie: JSESSIONID=ABC123 ----->| looks up session ABC123, adds item
|<-- Response -----|
| |
|--- Request 3 (checkout) |
| Cookie: JSESSIONID=ABC123 ----->| looks up session ABC123, sees cart contents
```

3. Code Example

```
protected void doGet(HttpServletRequest request, HttpServletResponse response) throws IOException {
    HttpSession session = request.getSession(); // creates a NEW session if none exists yet

    Integer visitCount = (Integer) session.getAttribute("visitCount");
    if (visitCount == null) {
        visitCount = 0;
    }
    visitCount++;
    session.setAttribute("visitCount", visitCount);

    response.getWriter().println("You've visited this session " + visitCount + " times.");
}
```

4. Key `HttpSession` Methods

Method	Purpose
<code>request.getSession()</code>	Returns the current session, CREATING a new one if none exists
<code>request.getSession(false)</code>	Returns the current session, or <code>null</code> if none exists (does NOT create one)
<code>setAttribute(name, value) / getAttribute(name)</code>	Store/retrieve session-scoped data
<code>invalidate()</code>	Destroys the session immediately (e.g., on logout)
<code>setMaxInactiveInterval(seconds)</code>	Configures how long a session can sit idle before automatic expiry

5. Real Life Analogy

- **Restaurant:** `HttpSession` is like a coat-check ticket — on your first interaction (checking your coat), you get a numbered ticket (Session ID). Every time you return to the counter, you show that SAME ticket, and the staff retrieves YOUR SPECIFIC coat (session data) rather than starting from scratch or mixing you up with another customer.

6. Edge Cases

Case	Result
Browser has cookies disabled	Session tracking typically falls back to URL rewriting (<code>; sessionId=ABC123</code> appended to URLs) if the server supports it, or session tracking fails entirely
Session left idle beyond <code>setMaxInactiveInterval</code>	Server automatically invalidates it — subsequent requests get a brand-new, empty session
Multiple server instances (load-balanced) without shared session storage	A user's session data may be "lost" if a subsequent request lands on a DIFFERENT server instance that doesn't have that session in its own memory — solved via "sticky sessions," a shared session store (Redis), or stateless authentication (JWT, covered later)

7. Common Mistakes

- Storing excessively large objects in the session, bloating server memory across potentially thousands of concurrent users' sessions.
- Forgetting to call `invalidate()` on logout, leaving a stale, still-valid session accessible if the session ID were somehow compromised.
- Not accounting for session replication/sharing needs in load-balanced, multi-server deployments.

8. Best Practices

- Keep session data minimal — store only small identifiers (like a user ID), not large objects, re-fetching full data as needed.

- Always `invalidate()` sessions explicitly on logout.
- In modern, horizontally-scaled microservice architectures, consider STATELESS authentication (JWT tokens, covered in the Spring Security topic) instead of server-side sessions, to avoid the multi-server session-sharing problem entirely.

9. Production Usage

- Classic server-rendered web applications (JSP-based, traditional Spring MVC with server-side rendering) rely heavily on `HttpSession` for login state and shopping-cart-style workflows.
- Modern distributed/microservice architectures increasingly favor stateless, token-based authentication (JWT) specifically to AVOID the session-sharing complexity across multiple server instances — a key architectural trade-off covered in depth later in this course.

10. Frequently Asked Interview Questions

- 1 **Why is session management needed, given HTTP is stateless?** To let a server remember a specific user's context (login, cart) across multiple independent, otherwise-unrelated HTTP requests.
 - 2 **How does the server know which `HttpSession` belongs to which browser?** Via a unique Session ID, typically transmitted through a cookie (`JSESSIONID`) that the browser automatically sends back on every subsequent request.
 - 3 **What happens if a user's browser has cookies disabled?** Session tracking may fall back to URL rewriting (embedding the session ID directly in URLs) if supported, or fail to track sessions at all.
 - 4 **What's a challenge with `HttpSession` in a load-balanced, multi-server deployment?** A user's session data may not be available if a later request is routed to a DIFFERENT server instance — solved via sticky sessions, shared session stores, or stateless token-based authentication.
 - 5 **How do you explicitly end a session (e.g., on logout)?** Call `session.invalidate()`.
-

TOPIC 5: COOKIES

1. Introduction

What is it? A cookie is a small piece of data the SERVER asks the CLIENT (browser) to store and send back with every subsequent request to that same domain — the underlying mechanism that makes `HttpSession` tracking (and many other features like "remember me" or user preferences) possible.

2. How Cookies Work

```
SERVER Response Header: Set-Cookie: username=alice; Max-Age=3600; Path=/
|
v
BROWSER stores this cookie, and automatically includes it on FUTURE requests:

CLIENT Request Header: Cookie: username=alice
```

3. Code Example — Creating and Reading Cookies

```
protected void doGet(HttpServletRequest request, HttpServletResponse response) throws IOException {
    // CREATE a cookie
    Cookie cookie = new Cookie("username", "alice");
    cookie.setMaxAge(60 * 60); // expires in 1 hour (in seconds)
    cookie.setPath("/"); // available across the entire application
    response.addCookie(cookie);

    // READ existing cookies
    Cookie[] cookies = request.getCookies();
    if (cookies != null) {
        for (Cookie c : cookies) {
            if (c.getName().equals("username")) {
                System.out.println("Found cookie: " + c.getValue());
            }
        }
    }
}
```

4. Key Cookie Attributes

Attribute	Purpose
Max-Age / Expires	How long the cookie persists (omitting both makes it a "session cookie," deleted when the browser closes)
Path	Which URL paths the cookie is sent for
Domain	Which domain(s) the cookie applies to
Secure	Cookie only sent over HTTPS connections
HttpOnly	Cookie inaccessible to client-side JavaScript — a critical security measure against XSS-based cookie theft
SameSite	Controls whether the cookie is sent on cross-site requests — a modern CSRF mitigation

5. HttpSession vs Cookies — How They Relate

Aspect	Cookies	HttpSession
Where data lives	Stored on the CLIENT (browser)	Stored on the SERVER
Size limit	Small (a few KB per cookie, limited count per domain)	Can hold larger, more complex Java objects
Relationship	The JSESSIONID cookie is HOW the browser tells the server which HttpSession belongs to it	Relies on a cookie (or URL rewriting) to be trackable across requests at all

6. Real Life Analogy

- **Bank:** A cookie is like a small membership card the bank gives you to carry — it doesn't contain your full account details (which stay securely in the bank's own records/server), just enough identifying information for the bank to recognize you next time.

7. Edge Cases

Case	Result
Cookie without <code>Max-Age/Expires</code> set	Becomes a "session cookie" — deleted automatically when the browser is closed
<code>HttpOnly</code> cookie	Cannot be read/modified via client-side JavaScript (<code>document.cookie</code>) — a deliberate security protection
Storing sensitive data (passwords, full credit card numbers) directly in a cookie	A serious security anti-pattern — cookies are visible/inspectable client-side unless properly protected, and should never hold sensitive raw data

8. Common Mistakes

- Storing sensitive information directly in cookies without encryption/signing.
- Forgetting to set `HttpOnly` on cookies that don't need JavaScript access, unnecessarily widening the XSS attack surface.
- Not setting `Secure` on cookies in HTTPS-only applications, risking transmission over an insecure connection.

9. Best Practices

- Mark cookies `HttpOnly` unless client-side JavaScript genuinely needs to read them.
- Mark cookies `Secure` in any HTTPS-serving application.
- Never store sensitive raw data directly in a cookie — use it only to reference server-side state (like a session ID) or store properly signed/encrypted tokens.

10. Frequently Asked Interview Questions

- 1 **What is a cookie?** A small piece of data the server asks the browser to store and automatically resend with future requests to the same domain.
- 2 **How does `HttpSession` actually work "under the hood" across requests?** Via a `JSESSIONID` cookie that the browser sends back on every request, letting the server look up the corresponding server-side session data.
- 3 **What does the `HttpOnly` cookie attribute do?** Prevents the cookie from being accessed via client-side JavaScript, mitigating XSS-based cookie theft.

- 4 **What's the difference between a "session cookie" and a "persistent cookie"?** A session cookie has no `Max-Age/Expires` set and is deleted when the browser closes; a persistent cookie has an explicit expiration and survives browser restarts.
 - 5 **Should sensitive data be stored directly in a cookie?** No — cookies should reference server-side state or hold properly signed/encrypted tokens, never raw sensitive data.
-

TOPIC 6: FILTERS

1. Introduction

What is it? A Servlet `Filter` intercepts HTTP requests and responses BEFORE they reach a Servlet (and/or after the Servlet has processed them), enabling cross-cutting concerns (logging, authentication checks, compression, encoding) to be applied UNIFORMLY across multiple Servlets, without duplicating that logic inside each one.

2. The Filter Chain

```
Request -> [Filter A] -> [Filter B] -> [Filter C] -> Target Servlet
|
Response <- [Filter A] <- [Filter B] <- [Filter C] <-----+
```

Each filter can inspect/modify the request BEFORE passing it along (via `chain.doFilter()`), and inspect/modify the response AFTER the next filter/Servlet in the chain has finished — a genuine "wrapping" pattern.

3. Code Example — A Logging Filter

```
import jakarta.servlet.*;
import jakarta.servlet.annotation.WebFilter;
import java.io.IOException;

@WebFilter("/*") // applies to ALL requests
public class LoggingFilter implements Filter {

    @Override
    public void doFilter(ServletRequest request, ServletResponse response, FilterChain chain)
        throws IOException, ServletException {
        long start = System.currentTimeMillis();

        chain.doFilter(request, response); // CRITICAL - passes control to the NEXT filter/Servlet
        // forgetting this call breaks the ENTIRE request!

        long duration = System.currentTimeMillis() - start;
        System.out.println("Request processed in " + duration + "ms");
    }
}
```

4. Real Life Analogy

- **Airport security:** A Filter is like a security checkpoint EVERY passenger (request) must pass through BEFORE reaching their gate (the Servlet) — the checkpoint can inspect, log, or even reject passengers, uniformly, without each individual gate needing its own separate security logic.
- **Restaurant:** A Filter is like a standard quality-check station EVERY dish passes through before reaching the customer — applied uniformly to every dish (request) regardless of which specific chef (Servlet) prepared it.

5. Common Uses of Filters

Use Case	Example
Authentication/Authorization checks	Reject requests without a valid session/token before they reach any Servlet
Logging	Record request details, timing, for every request uniformly
Compression	Compress response bodies (GZIP)
Character encoding	Ensure consistent request/response encoding
CORS handling	Add Cross-Origin Resource Sharing headers uniformly

6. Edge Cases

Case	Result
Forgetting to call <code>chain.doFilter()</code>	The request STOPS at that filter — the target Servlet (and any subsequent filters) never executes at all
Multiple filters mapped to the SAME URL pattern	All execute, in a container-defined order (typically declaration order, or explicitly ordered via <code>@Order/web.xml</code> sequence)
A filter throws an exception	Propagates up, typically resulting in an error response, unless caught within the filter itself

7. Common Mistakes

- Forgetting to call `chain.doFilter()` — silently breaks the entire request pipeline, since control never reaches the intended Servlet.
- Placing business logic (rather than genuinely cross-cutting concerns) inside a filter — filters should handle UNIFORM, request-independent concerns, not specific business rules.
- Not considering filter ORDER when multiple filters have dependencies on each other (e.g., an authentication filter should generally run before a logging filter that wants to log the authenticated user).

8. Best Practices

- Always ensure `chain.doFilter()` is called (unless deliberately short-circuiting the request, e.g., rejecting an unauthenticated request with an error response instead).
- Use filters for genuinely cross-cutting, request-independent concerns — authentication, logging, encoding — not specific business logic.
- Be deliberate about filter ordering when multiple filters interact.

9. Production Usage

- Spring's entire Security filter chain (Spring Security, covered later in this course) is built as a SERIES of Servlet Filters — this exact mechanism, just heavily pre-configured and abstracted by the framework.
- CORS handling, request logging, and authentication token validation in modern REST APIs are almost universally implemented as filters (or Spring's conceptually equivalent `Interceptor`/filter-chain abstractions).

10. Frequently Asked Interview Questions

- 1 **What is a Servlet Filter?** A component that intercepts requests/responses before and/or after they reach a target Servlet, enabling cross-cutting concerns to be applied uniformly.
- 2 **What happens if a filter forgets to call `chain.doFilter()`?** The request pipeline stops entirely at that filter — the target Servlet (and any later filters) never executes.
- 3 **Name three common real-world uses of filters.** Authentication/authorization checks, logging, and CORS header handling (also: compression, character encoding).
- 4 **How does Spring Security relate to Servlet Filters?** Spring Security is implemented as a chain of Servlet Filters internally, applying authentication/authorization checks to every incoming request before it reaches application code.

TOPIC 7: LISTENERS

1. Introduction

What is it? A Servlet Listener is a component that reacts to LIFECYCLE EVENTS within a web application — application startup/shutdown, session creation/destruction, attribute changes — without needing to be explicitly invoked by application code; the container calls them automatically when the corresponding event occurs.

2. Key Listener Interfaces

Interface	Triggered When
<code>ServletContextListener</code>	The web APPLICATION itself starts up (<code>contextInitialized</code>) or shuts down (<code>contextDestroyed</code>)
<code>HttpSessionListener</code>	A session is CREATED (<code>sessionCreated</code>) or DESTROYED/invalidated (<code>sessionDestroyed</code>)
<code>ServletRequestListener</code>	An individual HTTP REQUEST begins or ends
<code>HttpSessionAttributeListener</code>	An attribute is added/removed/replaced in a session

3. Code Example — Application Startup/Shutdown Listener

```
import jakarta.servlet.*;
import jakarta.servlet.annotation.WebListener;

@WebListener
public class AppStartupListener implements ServletContextListener {

    @Override
    public void contextInitialized(ServletContextEvent event) {
        System.out.println("Application starting up - initializing resources (e.g., DB connection pool)...");
    }

    @Override
    public void contextDestroyed(ServletContextEvent event) {
        System.out.println("Application shutting down - releasing resources...");
    }
}
```

4. Code Example — Tracking Active Sessions (Common Real Use Case)

```
import jakarta.servlet.http.*;
import jakarta.servlet.annotation.WebListener;
import java.util.concurrent.atomic.AtomicInteger;

@WebListener
public class SessionCounterListener implements HttpSessionListener {
    private static final AtomicInteger activeSessions = new AtomicInteger(0);

    @Override
    public void sessionCreated(HttpSessionEvent event) {
        System.out.println("Active sessions: " + activeSessions.incrementAndGet());
    }

    @Override
    public void sessionDestroyed(HttpSessionEvent event) {
        System.out.println("Active sessions: " + activeSessions.decrementAndGet());
    }
}
```

5. Real Life Analogy

- **Restaurant:** A Listener is like an automatic doorbell chime that rings whenever a customer walks in (session created) or leaves (session destroyed) — the restaurant doesn't need to manually check the door constantly; the event triggers the reaction automatically.
- **Hospital:** A `ServletContextListener` is like the hospital's opening/closing procedures — certain setup (turning on equipment, staffing the front desk) and teardown (locking up, backup systems) happen automatically at open/close, without a specific patient interaction triggering them.

6. Filters vs Listeners — Comparison Table

Aspect	Filter	Listener
Triggered by	Every matching HTTP request/response	Specific LIFECYCLE events (app start/stop, session create/destroy)
Can modify the request/response?	Yes	No — purely reactive/observational
Typical use	Cross-cutting per-request concerns (auth, logging)	Resource initialization/cleanup, monitoring session counts

7. Edge Cases

Case	Result
An exception thrown inside <code>contextInitialized()</code>	Can prevent the application from starting up successfully, depending on the container's handling
Multiple listeners of the SAME type registered	All execute for the corresponding event, in registration order

8. Common Mistakes

- Trying to use a Listener where a Filter is actually needed (e.g., attempting to modify request content in a Listener — not possible, Listeners are purely reactive/observational).
- Performing expensive, blocking operations inside `contextInitialized()`, unnecessarily delaying application startup.

9. Best Practices

- Use `ServletContextListener` for application-wide resource initialization (connection pools, caches) and graceful cleanup on shutdown.
- Use `HttpSessionListener` for legitimate session-lifecycle-driven concerns (like active-user monitoring/analytics), not as a substitute for proper request-level logic (which belongs in Filters/Servlets).

10. Production Usage

- Spring Boot's own application startup/shutdown hooks (`ApplicationListener`, `@PostConstruct`/`@PreDestroy`) build on this exact same underlying concept, abstracted at a higher level.
- Real-time active-user dashboards in enterprise applications commonly use `HttpSessionListener` to track live session counts.

11. Frequently Asked Interview Questions

- 1 **What is a Servlet Listener?** A component that reacts automatically to specific lifecycle events (application startup/shutdown, session creation/destruction) without needing explicit invocation from application code.

- 2 **What's the difference between a Filter and a Listener?** A Filter intercepts and can MODIFY every matching request/response; a Listener purely REACTS to lifecycle events and cannot modify request/response content.
- 3 **Name two common ServletContextListener use cases.** Initializing shared resources (database connection pools) on startup, and releasing them gracefully on shutdown.
- 4 **What interface would you use to track how many sessions are currently active?** HttpSessionListener, incrementing/decrementing a counter in sessionCreated()/sessionDestroyed().

12. Revision Notes (Topics 3-7 Consolidated)

WEB APPLICATION BASICS – CHEAT SHEET

=====

WAR structure: WEB-INF/ (protected: classes, lib, web.xml) + public static resources
web.xml (classic) -> mostly replaced by annotations (@WebServlet etc, Servlet 3.0+)

SESSION MANAGEMENT: HttpSession tracks a user ACROSS stateless HTTP requests,
via a JSESSIONID cookie linking browser <-> server-side session object

COOKIES: small client-stored data, sent automatically on matching requests
HttpOnly -> blocks JS access (XSS protection)
Secure -> HTTPS only
Max-Age -> persistent; omitted -> session cookie (dies with browser)

FILTERS: intercept EVERY matching request/response (cross-cutting concerns:
auth, logging, CORS) - MUST call chain.doFilter() or the pipeline breaks

LISTENERS: react to LIFECYCLE events (app start/stop, session create/destroy)
- purely observational, cannot modify request/response
ServletContextListener -> app-wide start/stop
HttpSessionListener -> session create/destroy

13. Homework

- 1 Build a simple Filter that logs the URL and processing time of every request, and a Listener that logs application startup/shutdown messages.
- 2 Explain, in your own words, why forgetting chain.doFilter() in a Filter is such a serious bug, and how you'd notice it during testing.

FINAL SUMMARY — ADVANCED JAVA COMPLETE

This concludes the **Advanced Java** section — 7 topics:

- 1 Enterprise Java Overview — J2EE/Java EE/Jakarta EE history, multi-tier architecture, modern Spring Boot shift.
- 2 MVC Architecture — Model/View/Controller separation, request flow, anti-patterns.
- 3 Web Application Basics — WAR structure, web.xml, annotation-based configuration.
- 4 Session Management — HttpSession, stateless HTTP, the JSESSIONID mechanism.
- 5 Cookies — client-side storage, security attributes (HttpOnly, Secure, SameSite).

- 6 Filters — cross-cutting request/response interception, the filter chain.
- 7 Listeners — lifecycle event reactions (application and session scope).

These foundational Servlet-container concepts (sessions, filters, listeners, MVC) directly underpin everything covered later in this course's Servlets, JSP, and Spring MVC sections — you now have the conceptual groundwork to understand modern frameworks as evolutions of these exact ideas, not disconnected new concepts.

(END OF ADVANCED JAVA MASTER NOTES — ready to proceed to JDBC whenever you say "Next Topic.")